HACKATHON · IBM · 2026

Agents IA Autonomes

De l'expression du besoin à la solution complète, zéro intervention humaine.

Hagop Hannachian · Yanis Hammaci · Yacine Tamdrari · Lyes Challal

Mars 2026

CONTEXTE

Une industrie qui court vite… mais dans le mauvais sens

01

Les développeurs rencontrent des difficultés

8h

/ semaine gaspillées

✦ **Notre réponse : un système multi-agents autonome, spécialisé et fiable — couvrant l'intégralité du pipeline.**

CONTEXTE

Une industrie qui court vite…

mais dans le mauvais sens

01

Les développeurs rencontrent des difficultés

8h

/ semaine gaspillées

02

L'IA arrive…

41%

du code écrit par IA en 2025

✦ **Notre réponse : un système multi-agents autonome, spécialisé et fiable — couvrant l'intégralité du pipeline.**

CONTEXTE

Une industrie qui court vite…

mais dans le mauvais sens

01

Les développeurs rencontrent des difficultés

8h

/ semaine gaspillées

02

L'IA arrive…

41%

du code écrit par IA en 2025

03

La déception

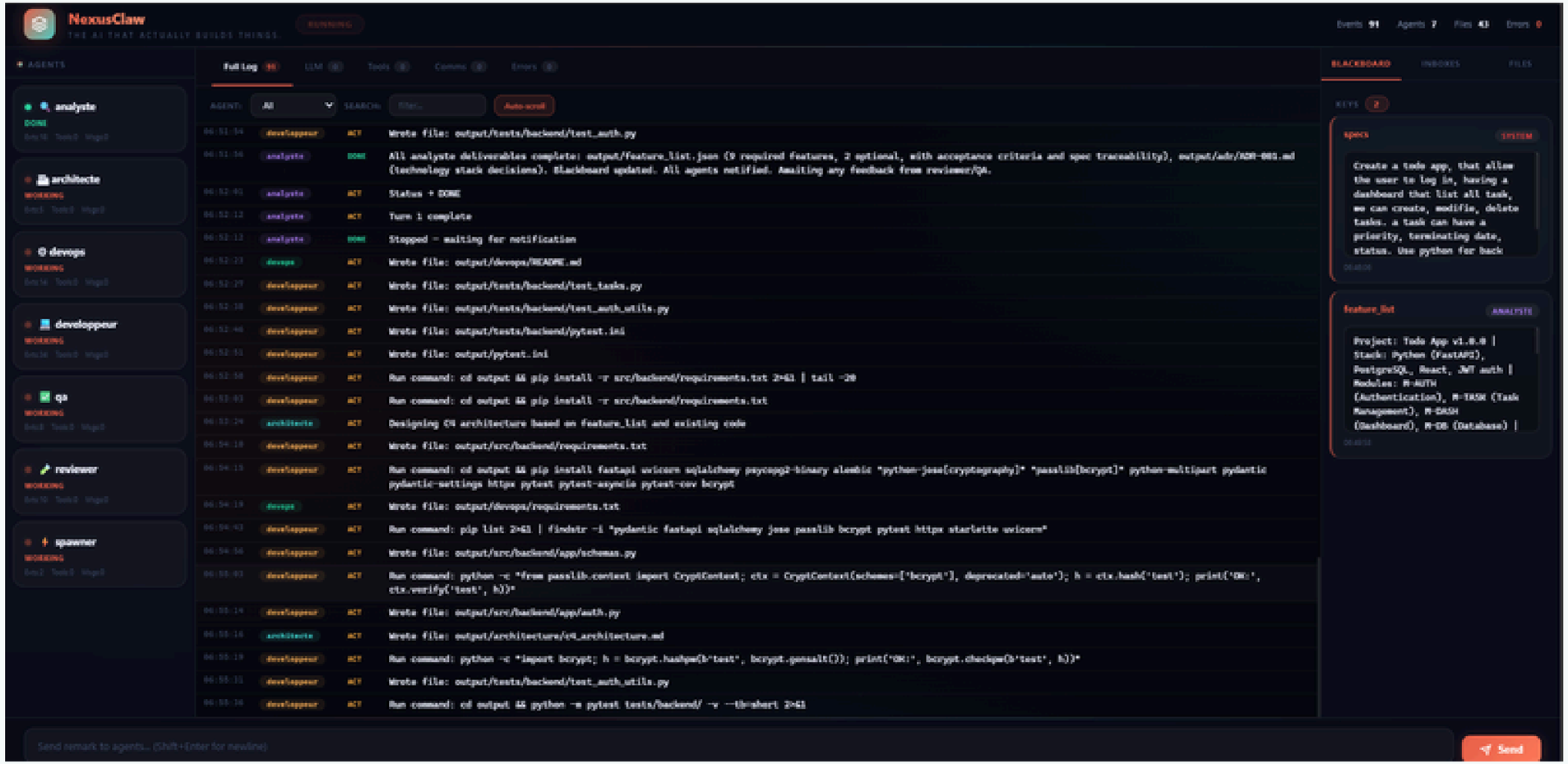
66%

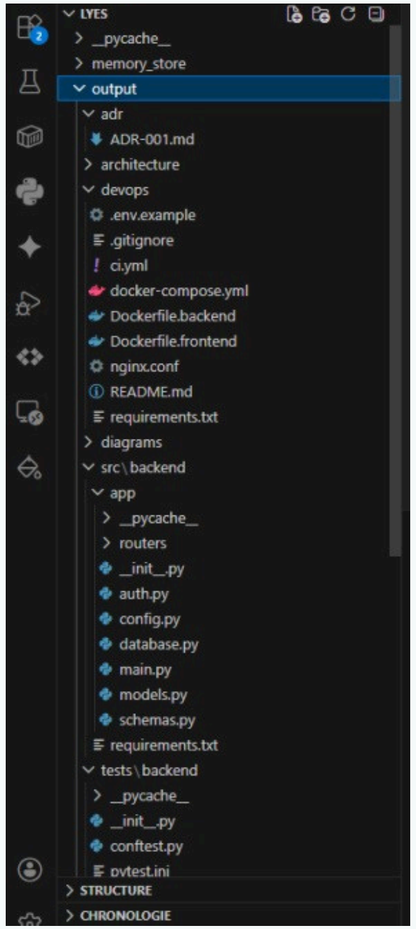
ne font plus confiance

✦ **Notre réponse : un système multi-agents autonome, spécialisé et fiable — couvrant l'intégralité du pipeline.**

Solution

Nexusclaw : Multi-agent autonome et très fiable





Solution

Nexusclaw : Basée sur des études récentes

arXiv:2602.16891v1 [cs.AI] 18 Feb 2026

OpenSage: Self-programming Agent Generation Engine

Hongwei Li[*1] Zhun Wang[*2] Qinrun Dai[3] Yuzhou Nie[1] Jinjun Peng[4] Ruitong Liu[3] Jingyang Zhang[5]
Kaijie Zhu[1] Jingxuan He[2] Lun Wang[6] Yangruibo Ding[7] Yueqi Chen[3] Wenbo Guo[1] Dawn Song[2]

Abstract

Agent development kits (ADKs) provide effective platforms and tooling for constructing agents, and their designs are critical to the constructed agents' performance, especially the functionality for agent topology, tools, and memory. However, current ADKs either lack sufficient functional support or rely on humans to manually design these components, limiting agents' generalizability and overall performance. We propose OpenSage, the first ADK that enables LLMs to automatically create agents with self-generated topology and toolsets while providing comprehensive and structured memory support. OpenSage offers effective functionality for agents to create and manage their own sub-agents and toolkits. It also features a hierarchical, graph-based memory system for efficient management and a specialized toolkit tailored to software engineering tasks. Extensive experiments across three state-of-the-art benchmarks with various backbone models demonstrate the advantages of OpenSage over existing ADKs. We also conduct rigorous ablation studies to demonstrate the effectiveness of our design for each component. We believe OpenSage can pave the way for the next generation of agent development, shifting the focus from human-centered to AI-centered paradigms.

1. Introduction

AI agents are under explosive growth, driven by their promising performance across diverse application domains (Wang et al., 2025a; Novikov et al., 2025; Ghafarollahi & Buehler, 2025; Ramos et al., 2025; Chu et al., 2025; Xi et al., 2025; Tang et al., 2026). This rapid evolution necessitates ef-

Figure 1. SageAgent (via OpenSage) vs. SOTA agents and ADKs on three popular agentic benchmarks. "G.3" refers to Gemini 3.

academia and industry have developed Agent Development Kits (ADKs) to provide the infrastructure and functionality required to construct agents equipped with complex tools and memory. A well-designed ADK is critical to both the utility and performance of the resulting agents, which are largely determined by three core architectural components: **agent topology, tooling system, and memory system**.

First, agent topology defines the agentic system's structure, including the architecture and tasks of individual agents, as well as their dependencies and interaction mechanisms. It forms the core of agents and directly determines their capability and effectiveness (Zhou et al., 2025; Qian et al., 2024). Second, agents interact with the environment through tools. A restricted toolset limits agents' retrieved information, frequently leading to hallucinations. It also constrains the agents' action spaces and the range of tasks they can accomplish (Li, 2025). Finally, memory systems enable agents to learn from past interactions, which is particularly helpful when executing complex tasks (Behrouz et al., 2025; Yan et al., 2025; Zhang et al., 2025; Suzgun et al., 2025).

SOTA ADKs, including Google ADK (Google, 2026), OpenHands SDK (Wang et al., 2024), OpenAI ADK (OpenAI, 2026), Claude ADK (Anthropic, 2026b), and LangChain (Inc. 2026), provide basic functionality but still

arXiv:2511.21380v1 [cs.SE] 26 Nov 2025

Multi-Agent Systems for Dataset Adaptation in Software Engineering: Capabilities, Limitations, and Future Directions

JINGYI CHEN*, The Hong Kong University of Science and Technology, China
XIAOYAN GUO*, University of Science and Technology of China, China
SONGQIANG CHEN, The Hong Kong University of Science and Technology, China
SHING-CHI CHEUNG†, The Hong Kong University of Science and Technology, China
JIASI SHEN†, The Hong Kong University of Science and Technology, China

Automating the adaptation of software engineering (SE) research artifacts across datasets is essential for scalability and reproducibility, yet it remains largely unstudied. Recent advances in large language model (LLM)–based multi-agent systems, such as GitHub Copilot's agent mode, promise to automate complex development workflows through coordinated reasoning, code generation, and tool interaction. This paper presents the first empirical study on how state-of-the-art multi-agent systems perform in dataset adaptation tasks. We evaluate Copilot, backed by GPT-4.1 and Claude Sonnet 4, on adapting SE research artifacts from benchmark repositories including ROCODE and LogHub2.0. Through a five-stage evaluation pipeline (file comprehension, code editing, command generation, validation, and final execution), we measure success rates, analyze failure patterns, and assess prompt-based interventions designed to enhance agent performance. Results show that current systems can identify key files and generate partial adaptations but rarely produce functionally correct implementations. Prompt-level interventions, especially providing execution error messages and reference code, substantially improve structural similarity to ground truth (from 7.25% to 67.14%), highlighting the importance of contextual and feedback-driven guidance. Our findings reveal both the promise and limitations of today's multi-agent LLM systems for dataset adaptation, and suggest concrete directions for building more reliable, self-correcting agents in future SE research.

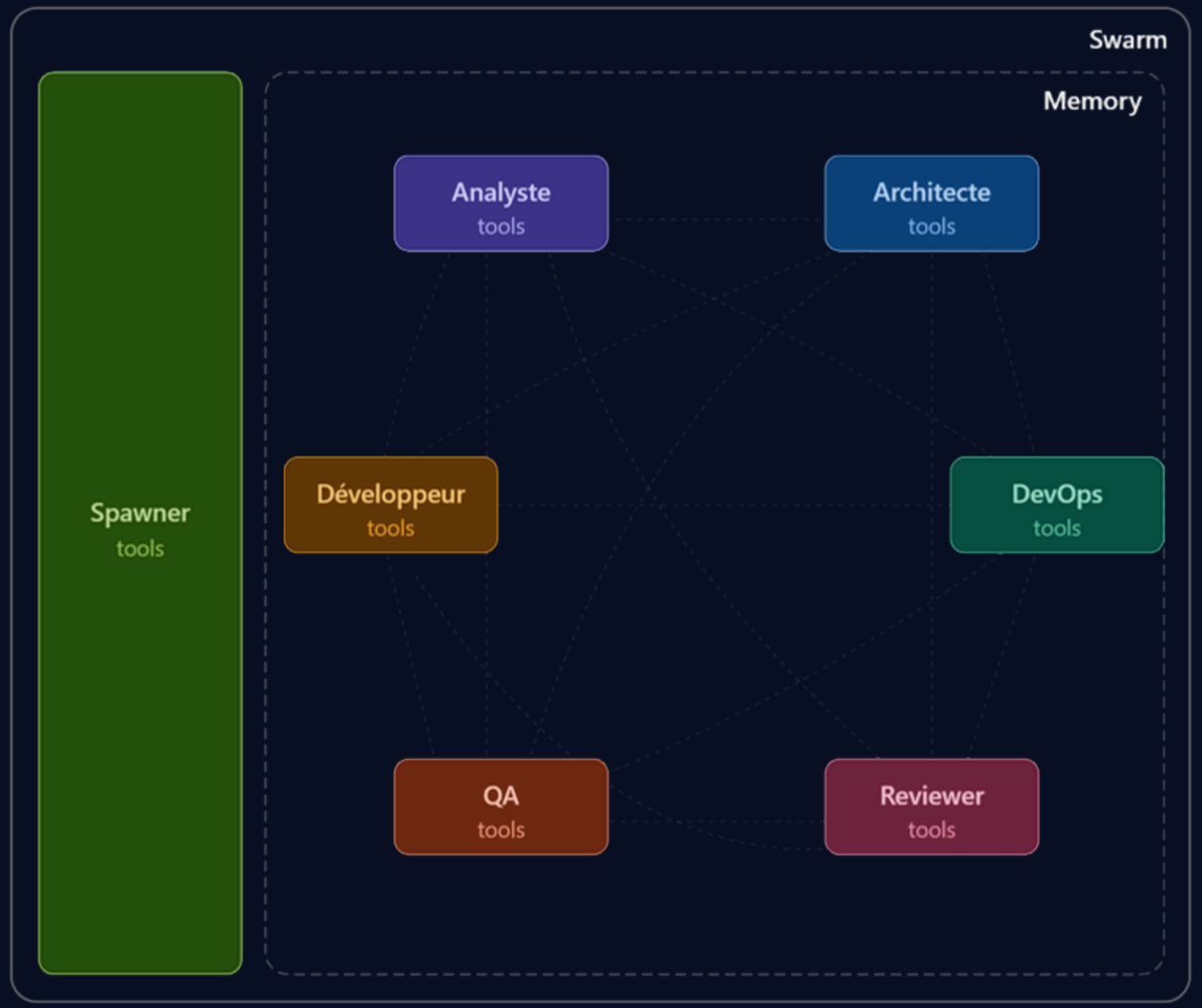
Additional Key Words and Phrases: Multi-Agent Systems, Dataset Adaption, Large Language Models, Automated Software Engineering

1 Introduction

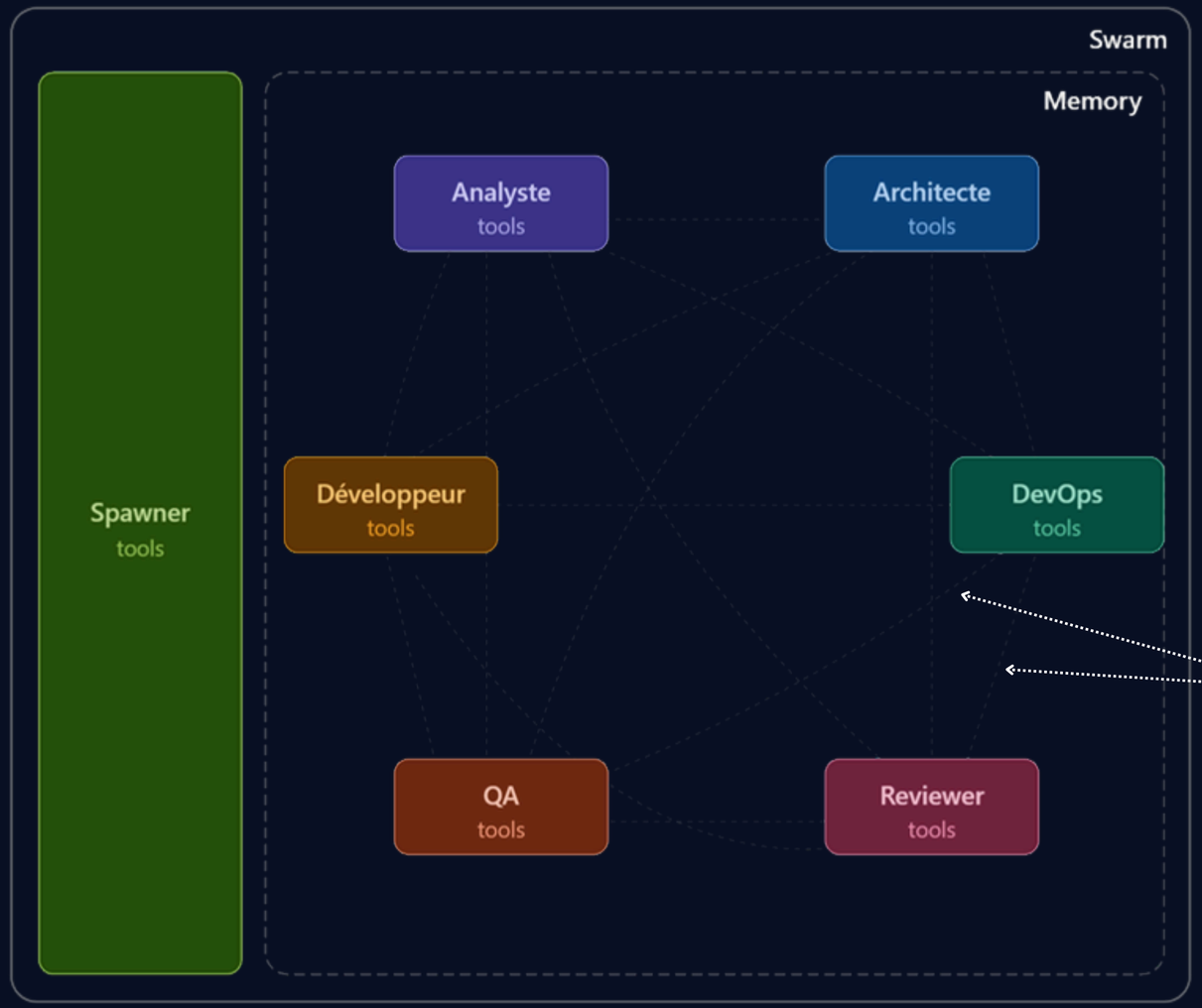
Software Engineering (SE) has entered a period of rapid innovation. In areas ranging from code generation and bug fixing to performance analysis and security, researchers are constantly proposing new techniques[9, 18, 27]. Each technique must be evaluated on multiple datasets to demonstrate its general effectiveness beyond a single repository or environment [8, 16]; yet automatic adaptation across datasets remains largely unexplored. Human experts can manually tailor a technique to each dataset (e.g., installing dependencies, building pipelines, and crafting test cases), but this approach does not scale as the number and diversity of datasets grow. The need for scalable, automated dataset adaptation is therefore critical for future SE research.

Large language models open a new possibility. Modern LLMs, such as ChatGPT and GitHub Copilot, can interpret code, search project directories, generate commands, modify files, and interact

Concrètement, comment ça marche ?

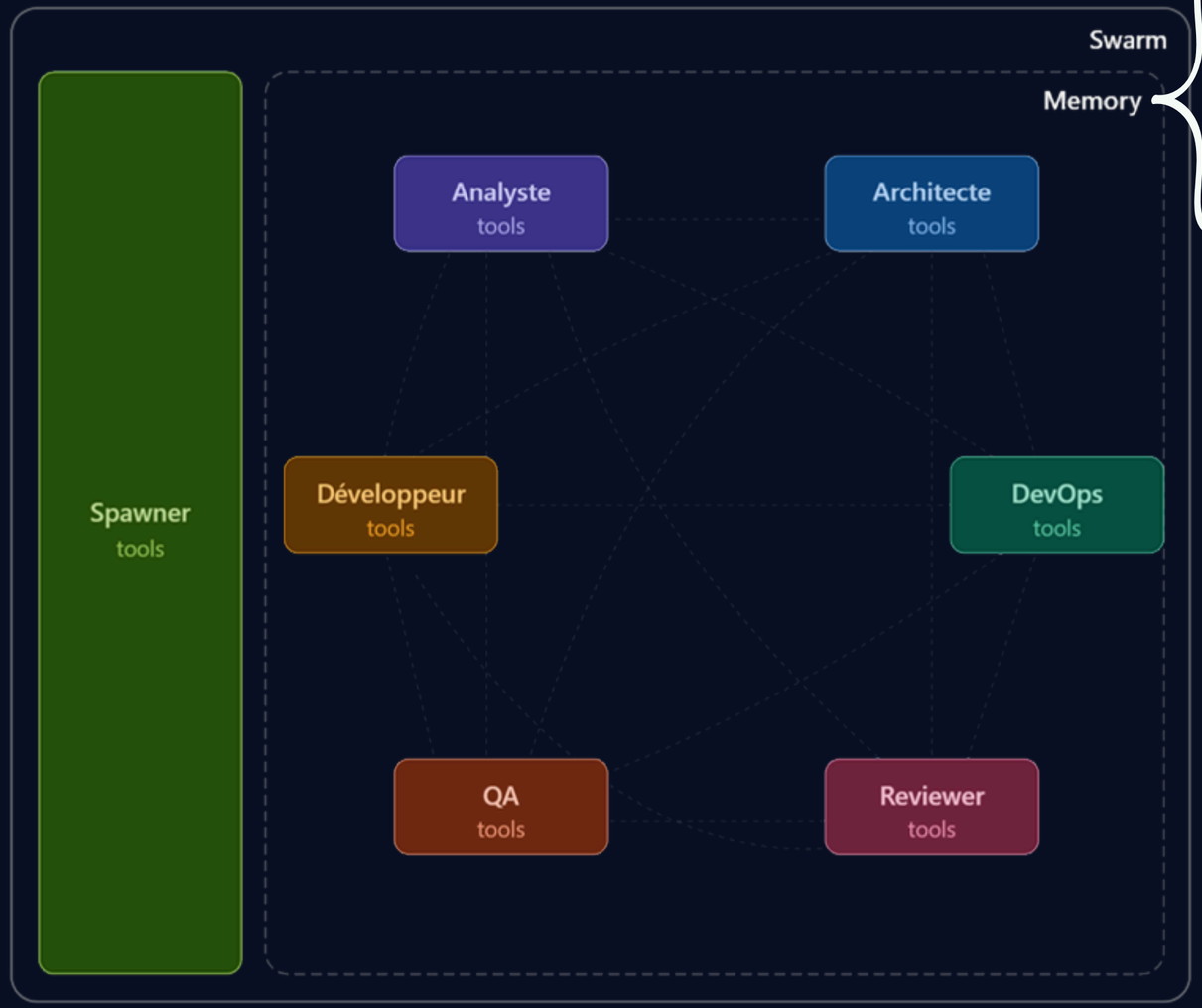


Concrètement, comment ça marche ?



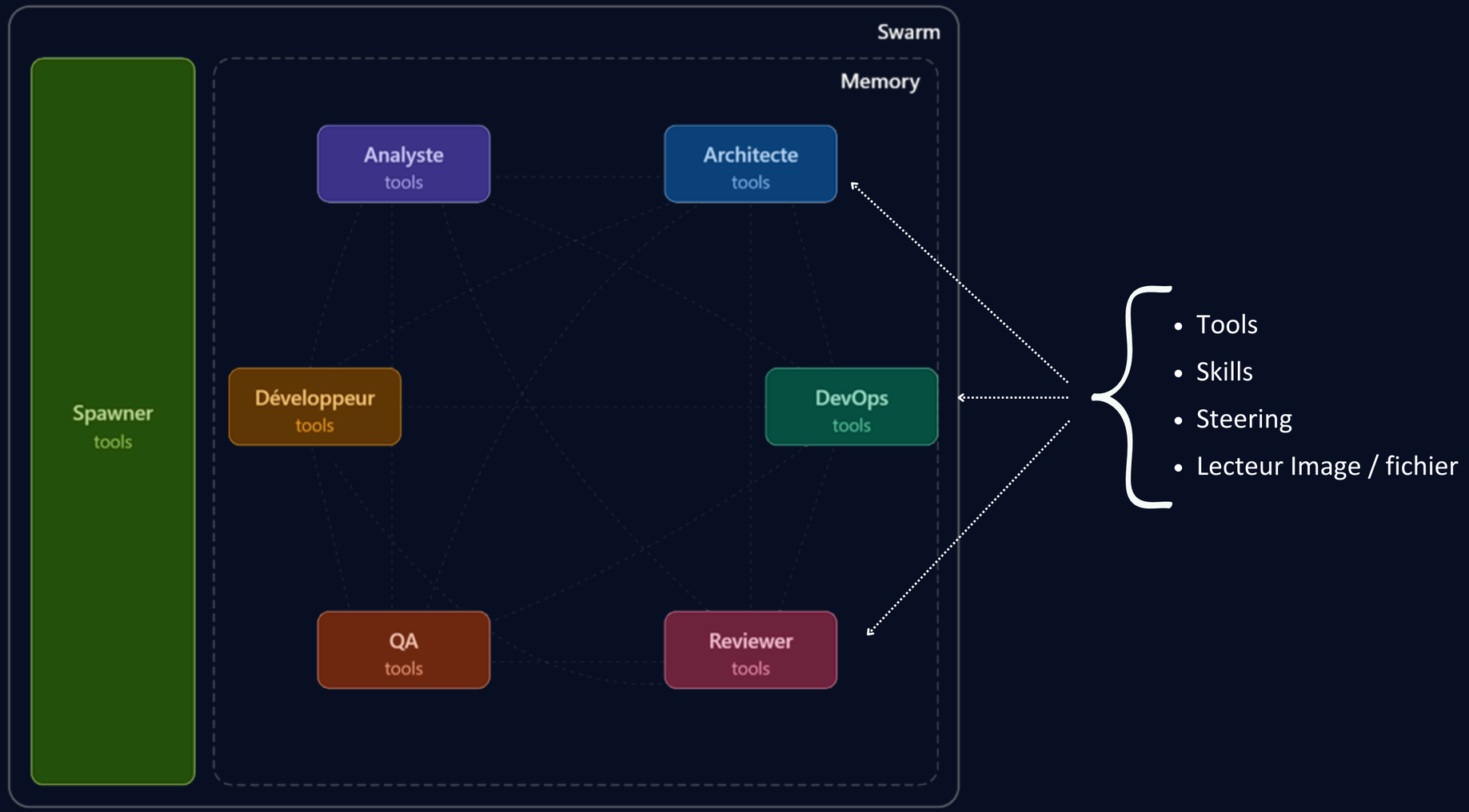
- Messagerie inter agent
- Blackboard
- Espace de travail commun

Concrètement, comment ça marche ?

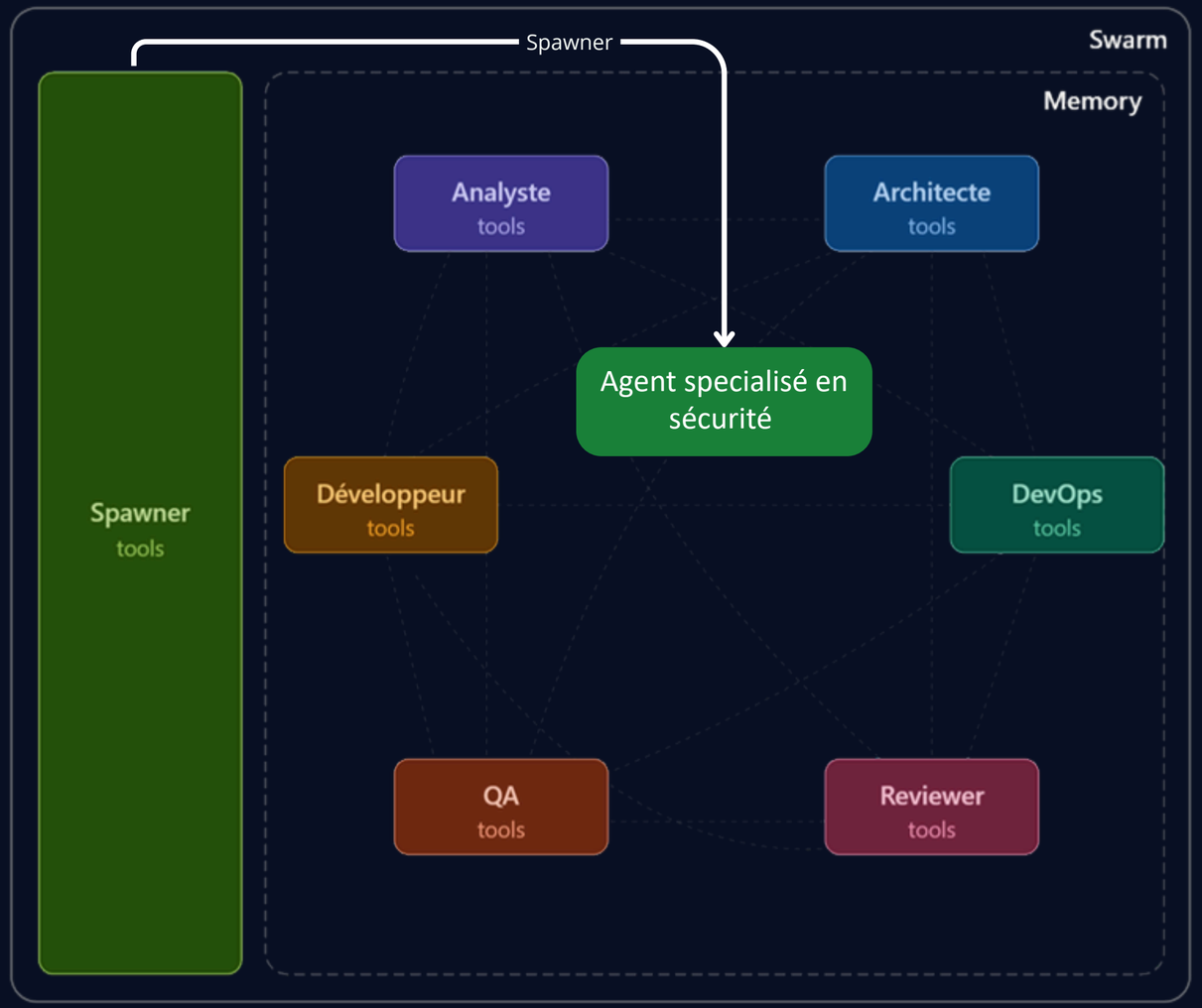


- Mémoire long terme hybride
- Indexation automatique des actions
- Réutilisation des solutions passées

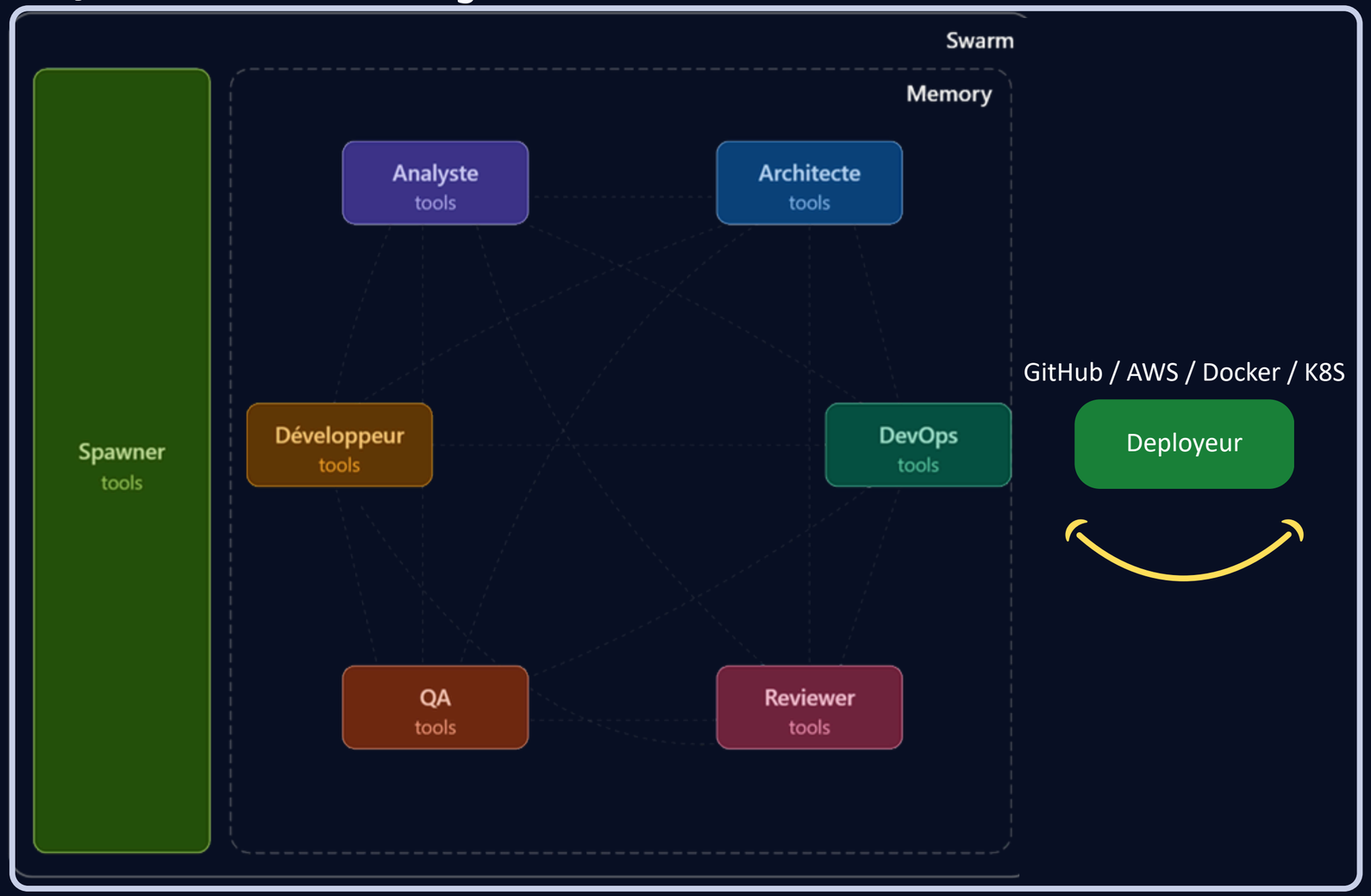
Concrètement, comment ça marche ?



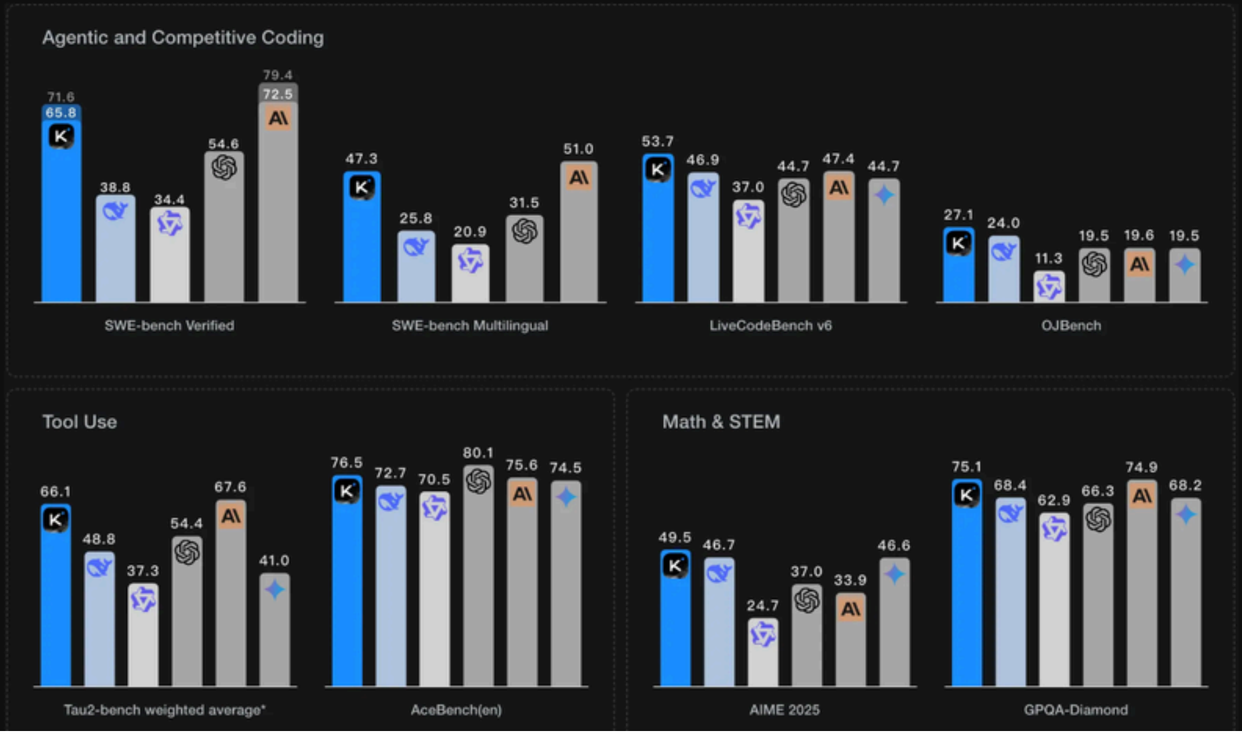
Concrètement, comment ça marche ?



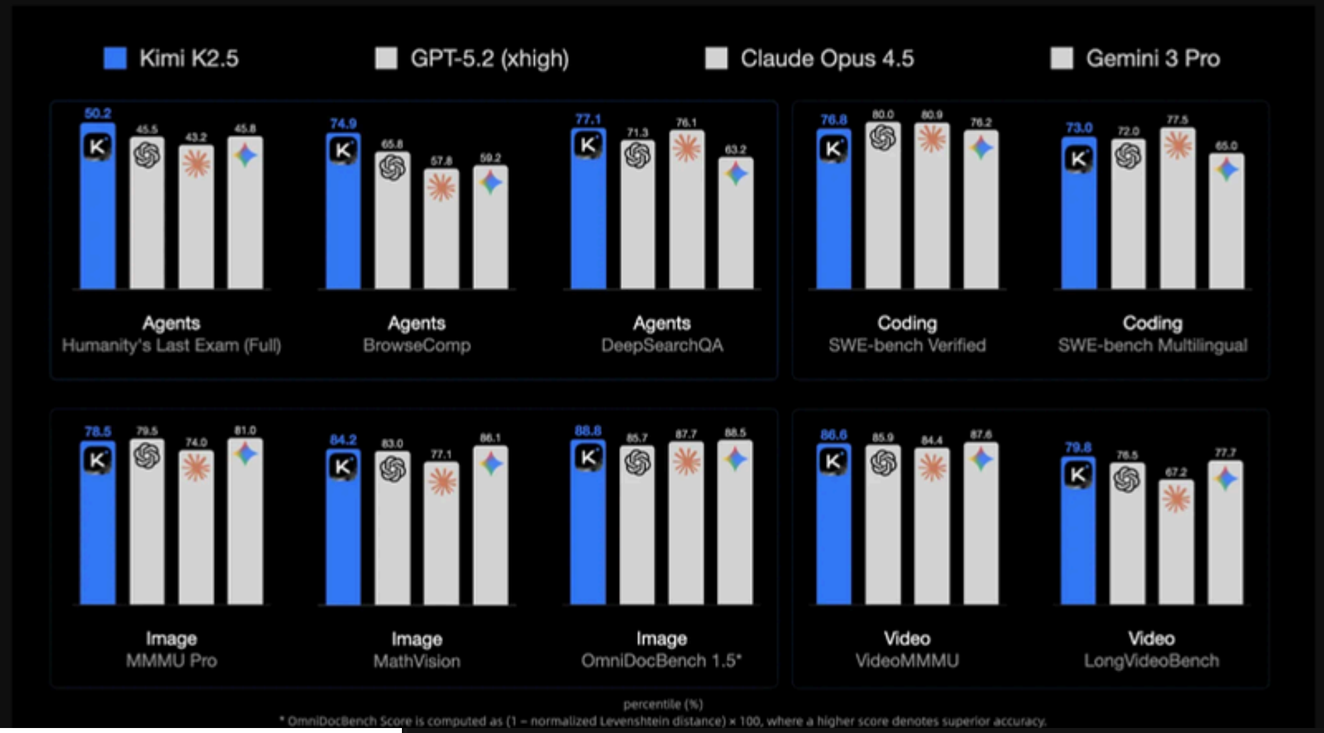
Concrètement, comment ça marche ?



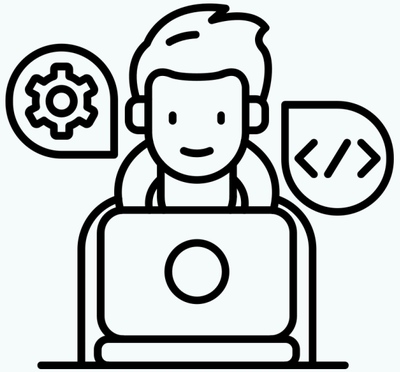
Kimi K2 is the new generation flagship model launched by Moonshot AI, featuring advanced agentic (autonomous reasoning and action) capabilities. Built with a 1T total parameter count and a 32B active parameter MoE (Mixture of Experts) architecture, Kimi K2 excels in AI coding and agent-building. Technical Report



Kimi K2.5 is Kimi's most intelligent model to date, achieving open-source SoTA performance in Agent, code, visual understanding, and a range of general intelligent tasks. It is also Kimi's most versatile model to date, featuring a native multimodal architecture that supports both visual and text input, thinking and non-thinking modes, and dialogue and Agent tasks. Tech Blog

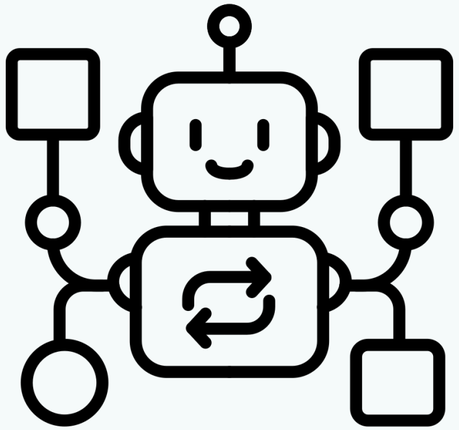


Combien ça coûte. Combien ça rapporte.



Équipe de 4 personnes

- Coût moyen par mois : 4 000€ × 4 = 16 000€
- Selon les statistiques (Stack Overflow) :
  40 % des tâches d'une équipe de dev sont répétitives
- Part de tâches répétitives (~40 %) :
  ≈ 6 400€/mois perdus



- Coûts d'infrastructure :
  10€/heure (infra + coûts LLM)
- Utilisation :
  ~160h/mois (équivalent temps plein)
- Calcul :
  10€ × 160h = 1 600€/mois

Gain ≈ 5000 €/mois

DÉMO

HACKATHON · IBM · 2026

Merci